SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

MLA0301 – REINFORCEMENT LEARNING

ASSIGNMENT 2 – CO2

INTELLIGENT AUTONOMOUS ROBOT NAVIGATION AND PATH OPTIMIZATION

Programme B.E. / B.Tech	
Course Code / Name	MLA0301 – Reinforcement Learning
Course Outcome	CO2
Maximum Marks	50
Nature / Duration	Individual Assignment / 1 Week
Name	Karthik V
Reg no	192425071
Github Link	MLA03_192425071_RL/Assignment 2 at main · KarthikMG-gf/MLA03_192425071_RL

CO2: Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks.

UNIT II – TABULAR METHODS AND Q-NETWORKS

Planning through Dynamic Programming and Monte Carlo – Control in Monte Carlo – Temporal Difference Learning Methods (TD(0), SARSA, Q-Learning) – Deep Q-Networks (DQN, DDQN, Dueling DQN, Prioritised Experience Replay).

Recommended Industry-Oriented Project Title

Design, Implement and Evaluate an Intelligent Autonomous Robot Navigation and Path Optimization System Using Dynamic Programming, Monte Carlo Methods, Temporal-Difference Learning and Deep Q-Networks

Industry Problem Context – Intelligent Smart Warehouse Robot Navigation

1. A large e-commerce warehouse handles thousands of products and customer orders every day. Autonomous mobile robots are deployed to transport products from storage locations to packing and dispatch stations. However, robots frequently encounter dynamic obstacles, blocked pathways, congestion, changing delivery priorities and battery limitations.

2. Traditional shortest-path algorithms cannot always provide an optimal long-term navigation strategy because the warehouse environment changes dynamically. The warehouse management system requires an intelligent decision-making agent that learns efficient navigation policies while minimizing travel distance, delivery time, energy consumption and collision risks.

3. The proposed system uses Reinforcement Learning techniques to develop an autonomous robot navigation strategy. Dynamic Programming methods are used to

evaluate and improve policies when the environment model is known. Monte Carlo methods learn from completed navigation episodes, while Temporal-Difference methods such as TD(0), SARSA and Q-Learning update value estimates through experience. Deep Q-Network approaches are considered when the state space becomes large.

4. **Design Challenge:** Develop an intelligent warehouse robot that learns an optimal navigation policy, avoids obstacles, minimizes transportation costs and reaches the destination efficiently under changing warehouse conditions.

Assignment Nature: Individual implementation and analytical evaluation. Bloom's Levels Assessed: K3 – Apply, K4 – Analyse and K5 – Evaluate.

Abstract

This report designs and evaluates a reinforcement-learning solution for autonomous robot navigation in a smart warehouse. The warehouse is represented as a grid Markov Decision Process. Dynamic Programming is applied when the model is known; Monte Carlo methods learn from completed episodes; TD(0), SARSA and Q-Learning update estimates from experience; and Deep Q-Network variants are discussed for large state spaces. A reproducible Python simulation compares Value Iteration, SARSA and Q-Learning on a 5 × 5 grid. The numerical outputs are generated by the supplied programs and represent a simplified simulation, not measurements from a physical warehouse.

Assumptions and scope

- Coordinates are (row, column); start = (0,0), goal = (4,4).
- Obstacles: (1,1), (1,3), (2,3), (3,1).
- The baseline environment is deterministic. An attempted move into an obstacle or outside the grid leaves the robot in place.
- Rewards: +20 for reaching the goal, −1 for a normal move and −10 for an obstacle/boundary attempt.
- The simulation does not model physical robot dynamics, battery use, multiple-robot traffic or moving obstacles. These are discussed as extensions.

TASK 1 – Model the Warehouse Navigation Problem Using Dynamic Programming

1.1 MDP formulation

The problem is an MDP M = (S,A,P,R,γ). A state is the robot's current traversable cell. The Markov property means that the current state contains all information needed to determine the probability distribution of the next state and reward; the full history is not required.

Element	Warehouse definition

State space S	Traversable grid cells (r,c); obstacle cells excluded; goal is terminal.
Action space A	Up, Down, Left, Right.
Transition P(s′|s,a)	Deterministic adjacent-cell transition; invalid/obstacle action keeps the robot in place.
Reward R	+20 goal; −1 ordinary move; −10 obstacle or boundary attempt.
Discount γ	0.95 in Task 5; 0.90 in the assignment's three-state example.
Terminal	Goal cell (4,4).

1.2 Labelled warehouse grid

```
S . . . .
. # . # .
. . . # .
. # . . .
. . . . G
```

S = start, G = goal, # = obstacle and . = free cell. The map is supplied in Dataset/warehouse_map.csv.

1.3 State-transition table

Current	Action	Next	Reward	Explanation
(0,0)	Right	(0,1)	−1	Normal move
(0,0)	Down	(1,0)	−1	Normal move
(0,1)	Down	(0,1)	−10	Obstacle at (1,1); no movement
(0,0)	Up	(0,0)	−10	Outside grid
(3,4)	Down	(4,4)	+20	Goal reached

1.4 Dynamic Programming and policy methods

Dynamic Programming solves an MDP when transition probabilities and rewards are known. Policy Evaluation estimates the value $V\pi(s)$ of a fixed policy using the Bellman expectation equation: $V\pi(s) = \Sigma a\pi(a|s)\Sigma s'P(s'|s,a)[R + \gamma V\pi(s')]$. Policy Improvement selects an action maximizing expected immediate reward plus discounted next-state value. Policy Iteration alternates evaluation and improvement until the policy is stable. Value Iteration repeatedly applies $V(s) \leftarrow \max a\Sigma s'P(s'|s,a)[R + \gamma V(s')]$ and combines evaluation and improvement in each backup.

For the robot, evaluation measures the long-term performance of its current route policy; improvement changes actions where another choice has a higher expected return. Policy Iteration is useful when a stable policy is desired, while Value Iteration directly computes optimal state values.

1.5 Reward structure and justification

Event	Reward	Reason
Destination reached	+20	Strong terminal success incentive.
Normal move	−1	Discourages wandering and long routes.
Obstacle/restricted entry	−10	Makes collision-risk actions undesirable.
Outside-grid action	−10	Penalizes invalid movement.

The goal reward is positive and larger than the movement cost, while obstacle and boundary penalties are severe. This encourages a short, safe route. Reward values are design assumptions and should be tuned against unwanted behaviours.

1.6 BFS versus iterative DP

Method	Time	Space	Objective
BFS	O(S+E), E≤SA; commonly O(SA)	O(S)	Fewest edges in an unweighted graph.
One DP sweep	O(SA)	O(S) values plus model	One Bellman backup per state/action.
K DP sweeps	O(KSA)	O(S) values plus model	Expected discounted reward / optimal policy.

BFS is suitable for a static unweighted shortest-path problem. DP is preferable when rewards, stochastic transitions and long-term consequences matter. BFS alone does not optimize a general reward function. The Markov property also requires including relevant variables such as battery, payload, traffic and dynamic obstacle status if they affect future transitions.

TASK 2 – Apply Policy Iteration, Value Iteration and Monte Carlo Methods

2.1 Five-state picking-robot MDP

State	Meaning
S0	Starting aisle
S1	Central aisle
S2	Product shelf / pickup
S3	Packing approach
S4	Packing station; terminal

Actions are Up, Down, Left and Right. An illustrative reward scheme is +8 for successful product collection, +20 for reaching the packing station, −1 for an ordinary move and −10 for an obstacle attempt. In a complete implementation, pickup status must be encoded in the state so that the same location before and after collection is distinguishable.

2.2 Policy Iteration

Initialize a policy, evaluate its state values with the Bellman expectation equation until the largest value change is below a threshold, and improve it by selecting the action with the highest expected return. Repeat evaluation and improvement until no action changes.

2.3 Completed Value Iteration program

For the assignment's three states, 0 = Start, 1 = Near Goal and 2 = terminal Goal. Actions are Forward and Stay. For each non-terminal state calculate $q(s,a)=r+\gamma V(s')$, append the action value, set V(s) to the maximum action value, and track $|V_{old}-V_{new}|$. Stop when the maximum change is below θ.

```
action_value = reward + gamma * V[next_state]
action_values.append(action_value)
V[s] = max(action_values)
delta = max(delta, abs(old_value - V[s]))
if delta < theta: break
```

Actual execution output from Code/task2_value_iteration.py:

```
Optimal State Values: [ 8. 10.  0.]
Optimal actions (0=Forward, 1=Stay): {0: 0, 1: 0}
```

At state 0, Forward gives −1 + 0.9(10) = 8, while Stay gives −2; therefore Forward is optimal. At state 1, Forward gives +10 and Stay gives −2. The optimal action is Forward in both non-terminal states.

2.4 Policy and discount factor explanations

- Policy evaluation estimates the quality of the current policy before changing it.
- Policy improvement chooses actions with higher expected returns and creates a better strategy.
- Value Iteration uses a maximum over actions in each update, combining evaluation and improvement.
- A larger discount factor gives greater weight to future rewards; a smaller value emphasizes immediate rewards.

2.5 Monte Carlo versus DP

Aspect	Dynamic Programming	Monte Carlo
Model	Requires transition and reward model.	No known transition model required.
Update	Bellman expectation/optimality backups.	Sampled returns from experience.
Timing	Can update without complete real episodes.	Standard return-based update waits for episode completion.
Complexity	K sweeps: O(KSA).	Episode length T: O(T) return calculation and O(T) storage.
Convergence	Standard finite-MDP conditions apply.	Improves with representative episodes; return variance can be high.

Monte Carlo return is $G_t = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-t-1} R_T$. It is not fully known until the episode ends, explaining why standard MC prediction waits for completed episodes.

TASK 3 – Monte Carlo Control and Temporal-Difference Learning

3.1 First-Visit and Every-Visit Monte Carlo

Monte Carlo prediction estimates V(s) by averaging returns following visits to state s. Returns are calculated backwards using $G_t = R_{t+1} + \gamma G_{t+1}$. First-Visit MC includes only the first occurrence of a state in each episode; Every-Visit MC includes every occurrence. First-Visit avoids repeated samples from the same episode, while Every-Visit uses more data but samples within an episode can be correlated.

3.2 Example return

For rewards +3 followed by +10 and $\gamma = 0.9$, the return from the first state is $G_0 = 3 + 0.9(10) = 12$; the return from the next state is $G_1 = 10$. Averages over completed episodes estimate state values.

3.3 Monte Carlo Control and ε-soft policy

Monte Carlo Control estimates Q(s,a) from completed episodes and improves the policy toward actions with higher returns. An ε-soft policy gives every action non-zero probability, for example choosing a random action with probability ε and a greedy action otherwise. This preserves exploration and helps discover alternative routes.

3.4 TD(0) numerical calculation

```
Vnew(St) = V(St) + α[R(t+1) + γV(St+1) − V(St)]
         = 5 + 0.2[3 + 0.9(8) − 5]
         = 5 + 0.2[3 + 7.2 − 5]
         = 5 + 0.2(5.2) = 6.04
```

The value rises from 5 to 6.04 because the reward plus discounted next-state estimate is greater than the old estimate. Future action selection may therefore favour actions leading to this state.

3.5 Monte Carlo versus TD(0)

Dimension	Monte Carlo	TD(0)
Experience	Complete episodes	Individual transitions
Bootstrapping	No; complete return	Yes; uses V(s′)
Update frequency	After episode termination	After each transition
Bias / variance	Often higher variance; complete sampled return	Bootstrapped target may be biased, often lower variance
Continuing operation	Needs episode boundaries/cut-offs	Can update online

3.6 Algorithm choice and complexity

TD(0) is a practical choice for continuing warehouse operations because it updates after every transition and does not retain an entire episode. First-Visit MC is useful for episodic delivery evaluation; Every-Visit MC can use more samples but repeated visits may be correlated. For T steps, MC return calculation and episode storage are O(T). Tabular TD(0) needs only the current transition and a value table, with O(1) additional work per transition.

Actual execution output:

```
First-visit MC values: {0: np.float64(9.2), 1: np.float64(12.0), 2: np.float64(10.0)}
Every-visit MC values: {0: np.float64(9.2), 1: np.float64(11.333), 2: np.float64(10.0)}
TD(0) updated value: 6.04
MC control action-value estimates: [-0.151  0.952  1.484  0.506]
Greedy action after learning: 2
```

TASK 4 – SARSA, Q-Learning and Deep Q-Networks

4.1 TD control equations

```
SARSA: Q(s,a) ← Q(s,a) + α[r + γQ(s’,a’) − Q(s,a)]
Q-Learning: Q(s,a) ← Q(s,a) + α[r + γ max_b Q(s’,b) − Q(s,a)]
```

α is the learning rate, r the immediate reward, γ the discount factor, s′ the next state and a′ the next action actually selected by the behaviour policy. Q-Learning uses the maximum estimated next-state action value.

4.2 Numerical calculations

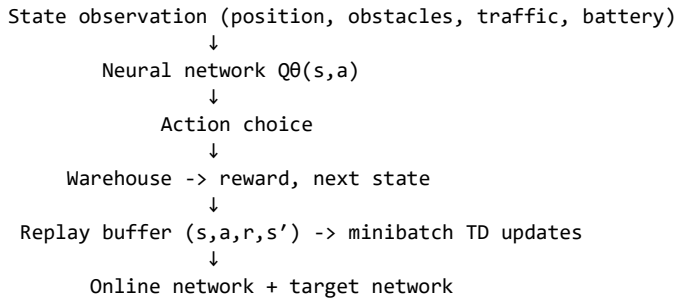
Parameter	Value
Current Q(s,a)	5
Reward	4
α	0.2
γ	0.9
Next SARSA action value	6
Maximum next-state value	8

```
SARSA = 5 + 0.2[4 + 0.9(6) − 5]
      = 5 + 0.2(4.4) = 5.88

Q-Learning = 5 + 0.2[4 + 0.9(8) − 5]
           = 5 + 0.2(6.2) = 6.24
```

SARSA is on-policy because it evaluates the next action actually selected by the behaviour policy. Q-Learning is off-policy because it uses the greedy maximum next action even if the behaviour policy explores. SARSA can reflect the cost of exploratory actions and is a reasonable safety-conscious baseline in a simulator. Q-Learning targets the greedy policy, but deployed exploration still requires explicit safety constraints.

4.3 DQN architecture and variants



- DQN approximates action values with a neural network and generalizes across large observations.
- Experience Replay stores transitions and samples minibatches to reduce sequential correlation and reuse data.
- Target Network is a delayed copy used to form more stable TD targets.
- DDQN selects the next action with the online network and evaluates it with the target network, reducing overestimation.
- Dueling DQN separates state-value and action-advantage streams before combining them.
- Prioritised Experience Replay samples transitions with larger TD errors more often; importance weights compensate for sampling bias.

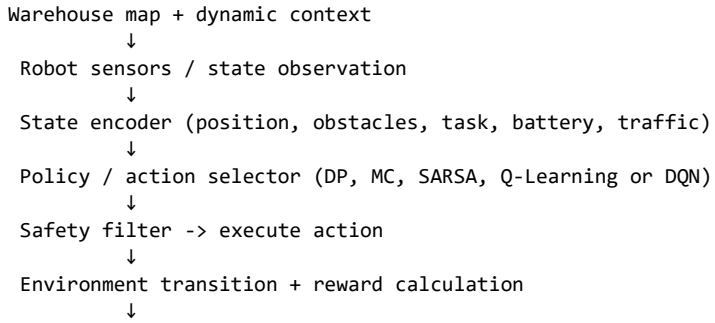
4.4 Tabular Q-Learning versus DQN

Aspect	Tabular	DQN
Storage	O(SA) table entries	O(P) network parameters, plus target network
Update cost	O(A) to find maximum and update entry	Forward/backpropagation over minibatches
Additional memory	Q-table	Replay buffer O(B × transition size), gradients, activations, optimizer state
Scalability	Poor for huge state spaces	Can generalize, but needs more data and compute
Interpretability	Individual entries inspectable	Neural representation less transparent

P is the number of trainable parameters. Comparing SA with P alone understates DQN cost because training also stores replay samples, gradients, activations and optimizer state.

TASK 5 – Integrated Intelligent Warehouse Navigation System

5.1 Architecture and integration



```
Learning update / replay buffer / policy evaluation
        └──────────────> updated policy
```

DP supplies a model-based reference policy when the transition model is known. MC Control learns from completed delivery episodes. SARSA and Q-Learning update tabular action values from individual transitions. DQN is appropriate to investigate when the state includes traffic, battery, payload and obstacle observations and is too large for a table.

5.2 Algorithm selection

Algorithm	When useful
Value / Policy Iteration	Known compact model; offline planning and benchmark.
Monte Carlo Control	Episodic deliveries and no reliable transition model.
SARSA	Online learning where exploratory behaviour should influence the target.
Q-Learning	Greedy target policy learned from exploratory experience in a safe simulator.
DQN	Large/high-dimensional observations.
DDQN / Dueling / PER	Potential improvements for overestimation, state-value structure and rare/high-error samples.

5.3 Navigation pseudocode

```
Initialize Q(s,a) or neural Q network, parameters and safety rules
For each episode: reset robot; observe state s; select ε-greedy action a
  Repeat until goal or step limit:
    Execute safety-filtered action; observe r, s' and terminal flag
    SARSA: select a'; update toward r + γQ(s',a')
    Q-Learning: update toward r + γ max_b Q(s',b)
    If DQN: store transition; sample replay minibatch; update network
    Set s = s' and continue
Evaluate on separate episodes; save policy, metrics and configuration
```

5.4 Implementation configuration

Parameter	Value
Grid / start / goal	5×5 / (0,0) / (4,4)
Obstacles	(1,1), (1,3), (2,3), (3,1)
Reward	Goal +20; move −1; obstacle/boundary −10
γ / α / ε	0.95 / 0.15 / 0.15
Training	500 episodes per TD method; max 100 steps
Evaluation	100 greedy episodes
Random seed	42

Python and NumPy implement the grid, state/action representation, reward logic, Value Iteration, SARSA, Q-Learning, learned route and evaluation. Dataset files define the map and rewards. Outputs are stored as text, JSON and CSV.

5.5 Experimental comparison

The following results are actual outputs from the included run. Value Iteration uses the known deterministic model. SARSA and Q-Learning learn from sampled interaction and are evaluated greedily over 100 episodes. Results are specific to this map, seed, reward structure and training budget.

Algorithm	Success rate	Mean reward	Mean steps

Value Iteration	100.0%	13.00	8.00
SARSA	100.0%	13.00	8.00
Q-Learning	100.0%	13.00	8.00



Figure 1. Greedy evaluation success rate from the reproducible grid simulation.

All three methods reached the goal in this particular evaluation, with the same 8-step route and accumulated reward of 13. This reflects the simple deterministic map and selected training configuration; it does not establish that SARSA or Q-Learning always matches a planner. Training performance and convergence depend on exploration, learning rate, random seed and training duration.

5.6 Learned path output

```
S * * . .\n. # * # .\n. . * # .\n. # * * *\n. . . . G\nPath coordinates: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (3, 2), (3, 3), (3, 4), (4, 4)]
```

5.7 Critical analysis and industry recommendation

Tabular methods become expensive when the state combines location, battery, payload, traffic, task priority and obstacle configuration. DQN can approximate values over high-dimensional observations but requires more data and computation, can be unstable and is less interpretable. For a compact known map, Value Iteration is an effective benchmark. For online learning in a safety-sensitive simulator, SARSA with an explicit safety filter is a defensible baseline. For larger observations, DDQN with replay and a target network is a candidate; Dueling DQN and PER should be experimentally validated rather than assumed superior.

A hybrid approach is beneficial: use model-based DP to establish safe reference routes where the map is known, and model-free TD learning to adapt from changing experience. Conventional shortest-path algorithms are efficient for static graphs but need updated costs when congestion, blocked aisles or priorities change. RL can learn long-term trade-offs but requires training and safety controls.

5.8 Reproducibility and submission checklist

- Run the scripts listed in README.md and retain the generated outputs.
- Capture your own terminal screenshots if required by the instructor; no screenshot is fabricated in this package.
- Create the GitHub repository named MLA03-<RegisterNumber>-Assignment2 and add its URL.
- Enter your name and register number on the cover page.
- Be ready to explain assumptions, equations, reward values, algorithm differences and simulation limitations.

Conclusion

The report applies MDP-based planning, Monte Carlo prediction and control, TD(0), SARSA, Q-Learning and DQN concepts to warehouse navigation. The included simulation provides a reproducible comparison of model-based

planning and tabular TD control. A practical system can combine DP planning with online TD adaptation and a separate safety layer; deep value approximation is a candidate for larger state spaces.